UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

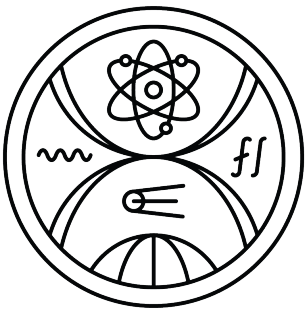


PROCEDURÁLNE GENEROVANIE ÚLOH S ALTERNATÍVNYMI RIEŠENIAMI V POČÍTAČOVÝCH HRÁCH

Diplomová práca

2026 Bc. Mária Cerulíková

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY



PROCEDURÁLNE GENEROVANIE ÚLOH S ALTERNATÍVNYMI RIEŠENIAMI V POČÍTAČOVÝCH HRÁCH

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Ing. Alexander Šimko, PhD.

Bratislava, 2026 Bc. Mária Cerulíková





Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Mária Cerulíková
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Procedurálne generovanie úloh s alternatívnymi riešeniami v počítačových hrách
Procedural generation of computer game quests with alternative solutions

Anotácia: Mnoho žánrov počítačových hier obsahuje úlohy (quests), ktoré hráč postupne rieši. Riešenie niektorých posúva hráča dopredu v príbehu hry, iné majú za cieľ naplniť herné prostredie obsahom a vytvoriť dojem plného a živého prostredia. Úlohy sú tradične dopredu navrhnuté a implementové v hre hernými vývojármi. Zvyšuje sa ale záujem o procedurálne generované úlohy. To umožňuje znižovať náklady na vývoj hry a poskytuje hráčovi neustále ďalší obsah na hranie. Súčasné algoritmy na procedurálne generovanie úloh sa obmedzujú na generovanie úloh, ktorú musí hráč vyriešiť presne predpísaným spôsobom.

Cieľ: Cieľom práce je navrhnúť algoritmus na generovanie úloh, ktoré môže hráč vyriešiť alternatívnymi spôsobmi. Uvažované budú úlohy, kde alternatívne spôsoby riešenia vedú k rovnakému koncu, ale aj rôznym koncom. Vyvinutý algoritmus bude otestovaný na referenčnom hernom svete. Súčasťou práce bude experimentálne vyhodnotenie vlastností algoritmu - kvalita generovaných úloh, časová zložitosť algoritmu a pod.

Literatúra:

J. Doran, I. Parberry. A prototype quest generator based on a structural analysis of quests from four MMORPGs. In Proceedings of the 2nd International Workshop on Procedural Content Generation in Games, PCGames '11, Bordeaux, France, 2011. ACM 2011, ISBN 978-1-4503-0872-4.

E. S. de Lima, B. Feijó, A. L. Furtado Procedural Generation of Quests for Games Using Genetic Algorithms and Automated Planning. 144-153. 18th Brazilian Symposium on Computer Games and Digital Entertainment, SBGames 2019, Rio de Janeiro, Brazil, 2019. IEEE 2019, ISBN 978-1-7281-4637-9.

V. Breault, S. Ouellet, J. Davies. Let CONAN tell you a story: Procedural quest generation. Entertainment Computing, Volume 38. Elsevier B.V. 2021.

Vedúci: Ing. Alexander Šimko, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.





Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Dátum zadania: 17.11.2025

Dátum schválenia: 17.11.2025 prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

Čestne vyhlasujem, že celú diplomovú prácu na tému Procedurálne generovanie úloh s alternatívnymi riešeniami v počítačových hrách, vrátane všetkých jej príloh a obrázkov, som vypracovala samostatne, a to s použitím literatúry uvedenej v priloženom zozname a nástrojov umelej inteligencie. Vyhlasujem, že nástroje umelej inteligencie som použila v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity.

.....

Bratislava, 2026 Bc. Mária Cerulíková

Poďakovanie

Abstrakt

Sem pôjde abstarkt...

Kľúčové slová: ...

Abstract

Here will go english translation of abstarct...

Keywords: ...

Obsah

1 Úvod **1**

2 Prehľad problematiky **2**

2.1 Generovanie úloh pomocou bezkontextovej gramatiky 2

2.2 Generovanie úloh pomocou plánovania 4

2.3 Generovanie úloh s využitím genetického algoritmu 7

2.4 Generovanie úloh s viacerými vetvami riešení 12

3 Ciele práce **17**

4 Metodika **18**

5 Návrh a implementácia **19**

6 Výskum **20**

7 Výsledky a diskusia **21**

8 Záver **22**

A Zdrojové kódy, dáta **24**

Zoznam obrázkov

2.1 Architektúra systému na procedurálne generovanie herných úloh využívajúceho genetického algoritmu 8
2.2 Grafické znázornenie genetického algoritmu, kde Φ je maximálny počet generácií a Θ je počet po sebe nasledujúcich úloh, ktoré majú byť vygenerované 9
2.3 Dejový oblúk s trojdejstvovou štruktúrou 10
2.4 Príklad štruktúry úlohy s vetviacim sa dejom s výškou H = 2 a vetviacim faktorom b = 3. Čiarkované kruhy a šípky predstavujú neplatné vetvy, ktoré nemajú riešenie. 12
2.5 Štruktúra deju hry reprezentovaná pomocou herného stromu 13
2.6 Príklad štruktúry rozvetvenej úlohy p 15

Zoznam tabuliek

Zoznam útržkov kódu

Terminológia

Skratky

-

Termíny

-

Kapitola 1

Úvod

...

Testovanie citácií: [1] [4] [6] [3] [2] [5]

Kapitola 2

Prehľad problematiky

Mnoho žánrov počítačových hier funguje na princípe, že pred hráča predkladá rôzne úlohy, ktorých plnením sa hráč posúva v deji alebo získava odmeny v podobe nových informácií, schopností a zdrojov. Tieto úlohy musia tradične herní vývojári dopredu navrhnúť a implementovať do hry. A keďže veľké hry môžu obsahovať aj stovky takýchto úloh, zvyšuje sa záujem o možnosti ich procedurálneho generovania.

Existuje viacero prác, ktoré navrhujú rôzne metódy procedurálneho generovania úloh. Najčastejšie sa v týchto prácach pozerá na hernú úlohu z pohľadu predikátovej logiky [1], [4], [2]. Pred začatím plnenia úlohy je herný svet v istom stave, ktorý vieme popísať pomocou predikátov (v hernom svete existujú nejaké postavy a predmety na určitých miestach, hráč vlastní isté predmety a má nejaké znalosti o hernom svete). Toto je počiatočný stav úlohy. Rovnako vieme pomocou predikátov povedať, čo hráč musí dosiahnuť, aby úlohu úspešne dokončil - cieľový stav. Potom na získanie presného postupu riešenia úlohy potrebujeme vyriešiť plánovací problém, ktorý nám vráti postupnosť akcií, ktoré musí hráč vykonať, aby úlohu úspešne dokončil a mohol tak získať odmenu. Iné prístupy vytvárania úloh zahŕňajú využitie bezkontextovej gramatiky [3] či veľkých jazykových modelov [6].

2.1 Generovanie úloh pomocou bezkontextovej gramatiky

Doran a Parberry [3] vo svojej práci analyzovali skoro 3000 herných úloh zo štyroch rôznych RPG hier. Prišli na to, že aj keď ľudský autori nemajú dané nijaké obmedzenia pri tvorbe herných úloh, tak štruktúra úloh, ktoré vytvárajú je podobná. Tieto úlohy síce môžu popisovať rozdielne herné prostredie, ale vzťahy medzi akciami ostávajú zachované. Toto pozorovanie ich viedlo k tomu, že zostavili množinu deviatich motivácií, pričom každá z analyzovaných úloh zapadá práve do jednej motivácie. Tieto motivácie sú: Poznanie, Pohodlie, Reputácia, Pokoj, Ochrana, Dobytie, Bohatstvo, Schopnosť

a Vybavenie. Motivácie majú zabezpečiť, aby úlohy, ktoré sú hráčovi prezentované prostredníctvom herných postáv pôsobili zámerne a nie náhodne generované. Každá nehráčska postava (NPC) má svoju motiváciu a úlohy, ktoré hráčovi zadáva vyplývajú z tejto motivácie. V priebehu hry sa motivácie jednotlivých NPC môžu meniť a to hlavne vtedy, keď pre nich hráč úspešne dokončil úlohu.

Ku každej motivácii prislúcha dve až sedem rôznych stratégií. Napríklad motivácia Poznanie má štyri stratégie: Doručiť predmet na štúdium, Vyzvedať, Rozprávať sa s NPC a Použiť predmet v teréne. Každá stratégia je tvorená sekvenciou niekoľkých akcií, ktoré hráč musí vykonať v danom poradí, aby úspešne dokončil úlohu. Každá akcia v stratégii môže byť buď jednoduchá atomická akcia, ktorú hráč vie rovno vykonať, alebo sa môže rekurzívne rozvíjať do postupnosti viacerých akcií. Štruktúra úloh sa potom dá vyjadriť vo forme bezkontextovej gramatiky, kde terminálmi sú atomické akcie. Jedna stratégia so svojou sekvenciou akcií môže vyzerať nasledovne:

$$\textit{Doručiť predmet na štúdium} : <\textit{get}><\textit{goto}>\ \textit{give}$$

Vidíme, že táto stratégia pozostáva z troch akcií: atomickej akcie *give* a neatomických akcií $<get>$ a $<goto>$. Tento zápis v podstate hovorí to, že ak má hráč za úlohu doručiť predmet na štúdium, musí najprv ten predmet získať ($<get>$), dostať sa k NPC, ktorému ho má odovzdať ($<goto>$) a potom predmet dá tomuto NPC (*give*). Keďže akcie $<get>$ a $<goto>$ nie sú atomické, budú sa ďalej rozvíjať podľa pravidiel bezkontextovej gramatiky. Napríklad, akcia $<goto>$ sa môže rozvinúť podľa troch pravidiel:

$$<\textit{goto}> ::= \varepsilon$$
$$<\textit{goto}> ::= \textit{explore}$$
$$<\textit{goto}> ::= <\textit{learn}>\ \textit{goto}$$

Prvé pravidlo hovorí, že hráč už je tam, kde má byť a preto nevykoná žiadnu akciu. Druhé pravidlo popisuje prípad, kedy hráč je v správnej časti herného sveta a očakáva sa od neho, že posúvaním svojej postavy preskúma svoje okolie. Tretie pravidlo rozoberá možnosť, že hráč potrebuje niekam ísť, ale nevie ako sa tam dostane. Preto to musí najskôr zistiť. Pomocou týchto pravidiel vieme rozvíjať každú stratégiu, až dokým nebudú všetky neterminály nahradené atomickými akciami. Vtedy budeme mať presnú postupnosť atomických akcií, ktoré hráč musí vykonať, aby úlohu úspešne dokončil.

Generovanie novej úlohy v tejto metóde vyzerá nasledovne. Ako vstupné parametre dostaneme motiváciu NPC, ktoré má úlohu zadať a inicializačnú hodnotu generátora náhodných čísel (ak je 0, tak sa použije systémový stav). Na základe zvolenej motivácie, generátor náhodne vyberie jednu z dostupných stratégií a potom začne vytvárať stromovú štruktúru, ktorá bude úlohu reprezentovať. Koreňom stromu je zvolená stratégia. Jej potomkovia sú akcie zo sekvencie akcií, ktoré ju rozvíjajú. Atomické akcie sú

listami stromu a vnútorné vrcholy sú tvorené neatomickými akciami, ktorých potomkov získame aplikovaním niektorého pravidla bezkontextovej gramatiky. Špecifické detaily úlohy (postavy, predmety, miesta) sú do úlohy dopĺňané náhodne počas toho, ako sa strom zostavuje. Aby sme potom z tohto stromu získali postupnosť atomických akcií, ktoré má hráč vykonať, musíme zostaviť zoznam listov pri prechode stromom v poradí preorder.

Táto metóda dokáže vytvoriť úlohy s dostatočne zložitou štruktúrou, ktorá môže obsahovať viaceré podúlohy, ktoré je potrebné splniť na úspešné dosiahnutie cieľa. Zároveň sa snaží zabezpečiť, aby vytvorené úlohy dávali zmysel pre postavy, ktoré ich hráčovi zadávajú. Realizuje to pomocou motivácií, ktoré zabezpečia, že postava zadá hráčovi len takú úlohu, ktorá ju vzhľadom na jej pozíciu v hre benefituje. Treba však dodať, že úlohy vytvorené touto metódou pozostávajú z presnej postupnosti akcií, ktorú hráč musí dodržať, inak úloha nebude úspešne dokončená a hráč nedostane odmenu. Inými slovami, neexistujú žiadne alternatívne riešenia jednej úlohy. Ďalším problémom je, že hoci metóda má dobre definované pravidla na rekurzívne rozširovanie akcií, nemá žiadne pravidlá na dosadzovanie špecifických detailov do týchto akcií. Ako špecifické detaily chápeme parametre akcií. Napríklad pre akciu $<goto>$ je špecifickáý detail, lokácia, kam treba ísť. Ako náhle pri tvorbe stromu je zistené, že treba dosadiť nový špecifický detail do akcie, metóda ho náhodne dosadí z dostupných preddefinovaných detailov. Neuvažuje o tom, či toto dosadenie je logické alebo či nespôsobí nekonzistenciu v hernom svete. Toto môže viesť k tomu, že predmet, ktorý bol po dokončení jednej úlohy umiestnený v lesa sa v nasledujúcej úlohe bez akéhokoľvek vysvetlenia ocitne v dedine. Na záver treba dodať, že výstupom z algoritmu je len textový popis akcií v človeku čitateľnej podobe a teda je potrebná manuálna implementácia takto vytvorenej úlohy do hry.

2.2 Generovanie úloh pomocou plánovania

Iný spôsob procedurálneho generovania herných úloh je predstavený v práci [1], kde sa na hľadanie úloh pozerá ako na plánovací problém. Plánovač potrebuje na vstup dostať doménu, ktorá obsahuje všetky akcie, ktoré v hernom svete môžu nastať a jeho počiatočný stav a cieľovú situáciu, ktorú chceme, aby bola dosiahnutá. Úlohou plánovača je potom nájsť postupnosť akcií, ktorá prepojí počiatočný stav sveta zo želaným cieľovým stavom. Táto postupnosť akcií predstavuje úlohu, ktorá by mohla byť prezentovaná hráčovi. Pri využití plánovača je však dôležité si uvedomiť, že samotný plánovací problém sa nezaoberá tým, aby výsledné úlohy pôsobili logicky a uveriteľne pre postavy, ktorých sa týkajú. Napríklad ak je požadovaný cieľový stav, aby monštrá boli zamknuté v klietke, nič nezabraňuje tomu, aby sa monštrá do klietky sami zavreli.

Aby autori zabezpečili, že správanie NPC bolo zmysluplné a pôsobilo zámerne, rozhodli sa reprezentovať každú postavu, ktorá môže zadávať úlohy (agenta), ako samostatný plánovací problém. Každý agent má priradenú svoju vlastnú tabuľku preferencií, v ktorej herný dizajnér môže nastaviť, ktoré akcie má tento agent preferovať a ktoré zase odmietať. Tieto preferencie sa odrazia na cenách jednotlivých akcií pri riešení plánovacieho problému. Základná cena všetkých akcií je dva. Ak však postava niektorú akciu preferuje, cena tejto akcie sa zníži o jedna. Ceny akcií, ktoré postava odmieta sa zvýšia na tri. Toto spôsobí, že morálne dobré postavy sa budú pri riešení svojich požiadaviek vyhýbať riešeniam, ktoré vyžadujú násilie a budú preferovať nenásilné riešenia. Na druhej strane, agresívnejšie postavy budú častejšie vyžadovať od hráča krádeže, vyzvedanie a zabíjanie.

Generovanie jednej úlohy v tomto systéme vyzerá nasledovne. V doméne je definovaných niekoľko predikátov s parametrami, ktoré môžu byť použité na popísanie cieľového stavu. Tieto predikáty môžu byť pomocou formátu PDDL zapísané napríklad takto: *{has?c?o}, {at?c?l}, {damaged?o}, ...*, kde *?c* je postava, *?o* je predmet a *?l* je miesto. Ako prvý je zvolený agent, pre ktorého sa úloha generuje. Systém náhodne vyberie niekoľko predikátov (ich počet je definovaný používateľom) a náhodne do nich dosadí parametre. Dosadené parametre musia byť správneho typu. Nemôže byť na miesto postavy dosadený predmet. Tieto predikáty sa stávajú potenciálnymi cieľmi úlohy. Pre každý takýto cieľ plánovač vytvorí plán na jeho dosiahnutie z aktuálneho stavu herného sveta. Zohľadňuje pri tom tabuľku preferencií zvoleného agenta. Získame tak množinu plánov P. Pre každý plán p_i z P, ktorý má cenu c_i sa vyráta priemerná cena jednej akcie:

$$avg_i = \frac{c_i}{|p_i|}$$

Ak teda platí, že cena akcie, ktorú agent preferuje je rovná jednej, potom čím je priemerná cena akcie v pláne bližšie k hodnote jedna, tým je celý plán vo väčšom súlade s preferenciami tohto agenta. Zo všetkých plánov je teda vybratý plán p^*, ktorého priemerná cena akcie sa najviac blíži k jednej a ten je prezentovaný hráčovi ako úloha.

$$p^* = \arg\min_{p_i \in P} |avg_i - 1|$$

Čím viac potenciálnych cieľov je na začiatku zvolených, tým je väčšia pravdepodobnosť nájdenia dobrého plánu pre zvoleného agenta. Avšak väčší počet cieľov zvyšuje počet výpočtov, ktoré musí plánovač vykonať.

Tento systém generovania úloh je súčasťou simulácie, v ktorej sa striedajú dve kolá - kolo NPC a kolo hráča. V kole NPC sa pre každú nehráčsku postavu spustí generovanie úlohy spôsobom spomínaným v predchádzajúcom odseku. Získané úlohy v podobe

postupnosti akcií sú transformované do čitateľného textu a príslušné postavy ich prezentujú hráčovi. Potom nasleduje kolo hráča. Vo svojom kole môže hráč komunikovať s NPC a prijať od nich úlohy. Prijatá úloha je prenesená na hráča (hráč ju má uloženú u seba a ostane mu aj keď NPC, ktoré mu ju zadalo si vygeneruje novú úlohu). Na záver svojho kola môže hráč prostredníctvom svojej postavy vykonať nejakú akciu, ktorá ovplyvní herný svet. Práve vykonaním takejto akcie má hráč šancu priblížiť sa k riešeniu niektorej úlohy, ktorá na neho bola prenesená. Potom čo je akcia vykonaná, nastane opäť kolo NPC. Stav herného sveta je aktualizovaný na základe zmien, ktoré hráč spravil a postavy si vytvoria pre hráča nové úlohy, ktoré už vychádzajú z tohto zmeneného stavu sveta.

Ľudskí tvorcovia herných úloh sú schopní vytvoriť úlohy prislúchajúce do každého z deviatich typov motivácie [3]. Preto sa autori tohto systému rozhodli overiť jeho kvalitu na základe toho, či je schopný vytvárať úlohy všetkých motivácií. Zostrojili klasifikačný modul, ktorý sa každej úlohe v podobe postupnosti akcií (plánu) pokúsi priradiť jednu z deviatich motivácií. Následne vygenerovali okolo tisíc úloh pre každé NPC v dvoch rôznych herných svetoch a pre každý herný svet osobitne vyhodnotili, koľko úloh každého typu motivácie systém vygeneroval. Výsledky ukázali, že systém je skutočne schopný generovať úlohy všetkých typov motivácií za predpokladu, že je poskytnutý herný svet dostatočne bohatý (obsahuje veľa postáv, predmetov a miest). V prípade druhého menšieho herného sveta systém vytvoril len úlohy prislúchajúce do štyroch rôznych typov motivácií, čím sa ukázalo, že kvalita vytvorených úloh nezávisí len od samotného systému, ale aj od kvality poskytnutej domény.

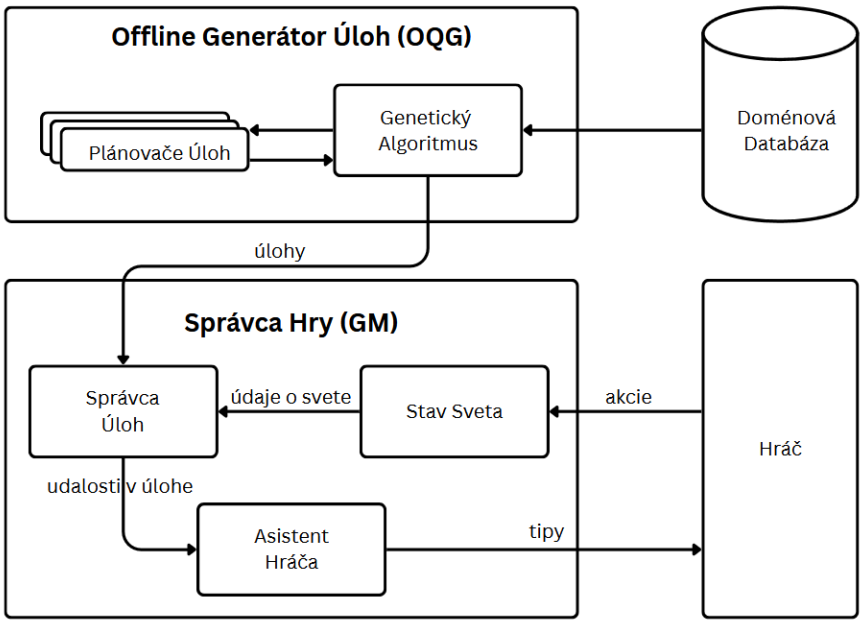
Výsledný systém nebol implementovaný do reálnej hry, ale bol odskúšaný pomocou textového rozhrania. V tomto rozhraní systém vypisuje aktuálne dostupné úlohy a hráč ich môže prijímať a zadávať akcie, ktoré chce vykonať. Po každej hráčovej akcii textové rozhranie aktualizuje dostupné úlohy. Výhodou tohto systému je, že dokáže generovať úlohy v reálnom čase. Po každej hráčovej akcii je vytvorená nová množia úloh, ktoré hráč môže plniť a teda hra hráčovi neustále predkladá nový obsah. Ďalšou výhodou je, že hra poskytuje hráčovi naraz niekoľko úloh, z ktorých si môže vybrať. Vďaka tomu môže hráč ovplyvňovať priebeh hry tým, že rozhoduje, ktoré úlohy splní a ktoré odmietne. Samotné úlohy sú však stále reprezentované, ako postupnosti akcií a hráč musí tieto postupnosti prísne dodržať, takže neexistujú alternatívne riešenia jednej úlohy. Takisto treba spomenúť, že nebolo vykonané žiadne testovanie, ktoré by overilo, či úlohy generované systémom sú pútavé a zábavné na plnenie, alebo či celkový príbeh vytvorený len pomocou úloh generovaných čisto na základe aktuálneho stavu sveta a akcií vykonaných hráčom je dostatočne zaujímavý.

2.3 Generovanie úloh s využitím genetického algoritmu

V práci [4] je navrhnutá metóda, ktorá sa snaží nájsť čo najkvalitnejšie úlohy s využitím genetického algoritmu a plánovania. Na rozdiel od prechádzajúcich prác, ktoré za úlohu považujú postupnosť akcií, v tejto práci je jedna úloha tvorená množinou zadaní, ktoré hráč musí splniť. Tieto zadania môžu byť napríklad: zabi nepriateľa, zozbieraj predmet, zachráň postavu. Samotná postupnosť akcií (plán) je v tejto práci označená ako dej (ang. plot) úlohy. Ďalším rozdielom v prístupe v tejto práci je, že sa nezameriava na NPC, ktoré úlohy zadávajú. Namiesto toho sa snaží, aby jednotlivé úlohy na seba nadväzovali a nevznikali medzi nimi logické rozpory. Ak teda v jednej úlohe hráč opravil most, nemôže to byť od neho znovu požadované pokiaľ v niektorej predchádzajúcej úlohe nebol most znovu zničený.

Keďže herné svety môžu obsahovať stovky postáv a tisícky predmetov a miest, nájsť „dobrú“ množinu zadaní pre úlohu v tomto obrovskom prehľadávacom priestore je veľmi zložitý optimalizačný problém. Preto sa autori rozhodli využiť genetický algoritmus, ktorý dokáže nájsť dobré riešenie efektívnejšie ako tradičné prehľadávacie metódy. Genetický algoritmus abstrahuje evolučné princípy ako „prirodzený výber“ a „prežitie najschopnejších“ a aplikuje ich na výpočtové procesy, ktoré môžu byť využité na nájdenie optimálneho riešenia problému. V tradičnej implementácii genetického algoritmu je najprv vytvorená počiatočná populácia jedincov (potenciálnych riešení problému), ktorá je vygenerovaná náhodne. Následne je každý jedinec ohodnotený funkciou fitness (zdatnosti). Táto funkcia preskúma vlastnosti jednotlivca a na ich základe mu priradí číselné hodnotenie. Čím vyššie hodnotenie, tým viac sa jedinec podobá na optimálne riešenie. Po vyhodnotení všetkých jedincov je niekoľko najlepších z nich zvolených za rodičov nasledujúcej generácie. Zo zvolených rodičov sa vždy vyberú dvaja a tí prejdú procesom kríženia (crossover) a mutácie (mutation). Pri krížení sa vyberú niektoré vlastnosti prvého rodiča a niektoré vlastnosti z druhého rodiča, čím vznikne nový jedinec - ich potomok. V proces mutácie môže byť potomkovi náhodne pridaná, odstránená alebo vymenená niektorá vlastnosť. Toto zabezpečí, aby algoritmus neostal zaseknutý v lokálnom optime a aby si nová generácia zachovala rozmanitosť. Proces kríženia a mutácie sa opakuje s rôznymi dvojicami rodičov, do kým nevznikne dostatok potomkov. Keďže potomkovia vznikali kombináciou vlastností dobrých jedincov z predchádzajúcej populácie, očakáva sa, že budú lepší ako ich rodičia. Táto nová generácia je potom opäť podrobená hodnoteniu a celý proces sa opakuje dokým nie sú splnené ukončovacie kritériá (zvyčajne vytvorenie daného počtu generácií). Po skončení je zvolený najlepší jedinec poslednej generácie a ten je považovaný za najlepšie riešenie problému.

Na obrázku 2.1 je znázornená architektúra navrhnutého systému. Môžeme vidieť, že

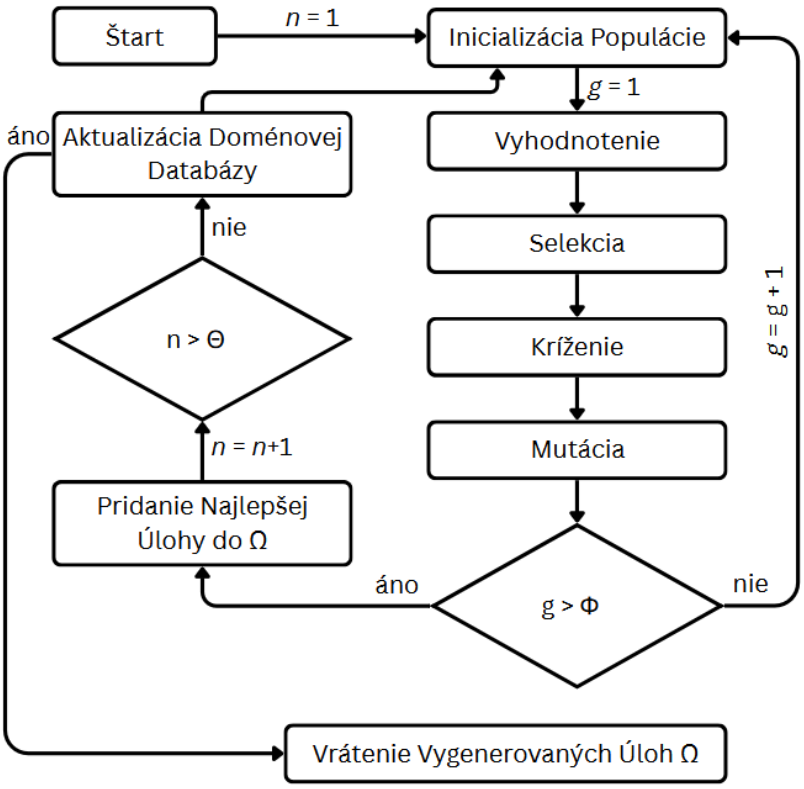


Obr. 2.1: Architektúra systému na procedurálne generovanie herných úloh využívajúceho genetického algoritmu
Zdroj: Preložené do slovenčiny podľa obr. 1 z [4].

generátor úloh pozostáva z dvoch podsystémov: Offline generátora úloh (Offline quest generator - OQG) a Správcu hry (Game manager - GM). OQG sa vykonáva vo fáze predspracovania (offline). Spúšťa genetický algoritmus a spracováva jeho výsledky (vygenerované úlohy). Zároveň zabezpečuje vykonávanie plánovača úloh, ktorý sa používa na overenie splniteľnosti jednotlivých úloh a ich ohodnotenie pomocou funkcie fitness. Úlohy sú generované na základe informácií uložených v doménovej databáze (presný popis doménovej databázy sa nachádza neskôr v texte). Výsledkom OQG je postupnosť nadväzujúcich úloh pre GM. GM beží počas samotnej hry a zabezpečuje interakciu s hráčom. Akcie hráča ovplyvňujú herný svet, čo využíva správca úloh na to, aby mohol sledovať hráčov postup pri plnení úloh. Súčasne asistent hráča poskytuje hráčovi tipy o cieľoch aktuálnej úlohy.

Na obrázku 2.2 je bližšie popísaný genetický algoritmus použitý v tomto systéme. Vidíme, že sa v ňom objavujú všetky kroky tradičného genetického algoritmu (inicializácia populácie, vyhodnotenie, selekcia, kríženie, mutácia). Zároveň je pridaný kontrolný cyklus, ktorého úlohou je zabezpečiť generovanie postupnosti úloh. Vždy, keď algoritmus nájde novú úlohu, pridá ju do postupnosti nadväzujúcich úloh Ω a skontroluje, či vygeneroval požadovaný počet úloh. Ak nie, aktualizuje doménovú databázu podľa zmien spôsobených v priebehu poslednej úlohy a vráti sa opäť na začiatok, aby vygeneroval ďalšiu úlohu.

V tomto genetickom algoritme je každý jednotlivec - jedna úloha formálne reprezentovaná ako plánovací problém:



Obr. 2.2: Grafické znázornenie genetického algoritmu, kde Φ je maximálny počet generácií a Θ je počet po sebe nasledujúcich úloh, ktoré majú byť vygenerované
Zdroj: Preložené do slovenčiny podľa obr. 2 z [4].

$$Q = (P, S_0, G, O)$$

P je množina atómov, ktoré môžu byť použité na popísanie stavu herného sveta. O je množina akcií, S_0 je počiatočný stav, v ktorom sa herný svet nachádza na začiatku úlohy a G je cieľový stav. P a O sú rovnaké pre všetkých jednotlivcov, ale S_0 a G sa v jednotlivých úlohách líšia a preto sú použité pri krížení a mutácii.

Jednotlivci počiatočnej populácie sa generujú náhodne na základe informácií definovaných v doménovej databáze DB.

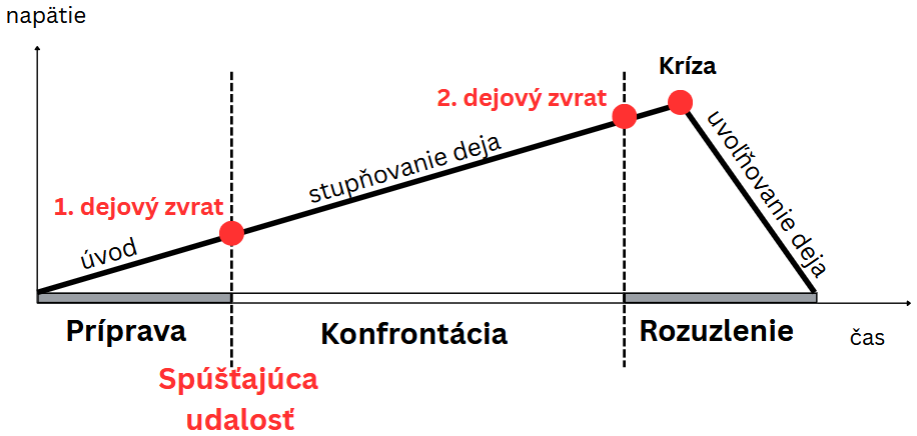
$$DB = (A, B, \Gamma, \Delta, E)$$

A je množina dvojíc $\alpha_i = (obj_i, objType_i)$, ktoré každému objektu v hernom svete priraďujú špecifický typ objektu (napr. $A = \{(john, character), (home, place), (sword, item)\}$). B je množina uzemnených literálov, ktoré popisujú vlastnosti a vzťahy objektov existujúcich v hernom svete (napr. $B = \{path(home, village), alive(john), at(john, home), ...\}$). Γ je množina obmedzení, ktoré zabezpečujú správne generovanie počiatočných a cieľových stavov úloh. Sú definované tri typy obmedzení. Prvý typ zabezpečuje dosadzovanie premenných správnych typov do predikátov. Druhý typ zabraňuje, aby v jednej úlohe

súčasne platili dva protichodné predikáty (napr. nemôže súčasne platiť $alive(john)$ a $dead(john)$). Tretí typ definuje existenčnú jedinečnosť (napr. jedna postava sa môže v jednom momente nachádzať len na jednom mieste). Δ je množina všetkých akcií a E je množina dvojíc $\varepsilon_i = (o_i, tension_i)$, ktoré každej operácii priraďujú jej efekt na napätie deja úlohy. Akcia môže napätie zvyšovať (+), znižovať (-) alebo zachovať (=).

Na vytvorenie počiatočnej populácie je potrebné vygenerovať plánovací problém pre každého jednotlivca. P je vytvorené z A z DB a B z DB a O je zostrojené z akcií v Δ z DB. Počiatočný stav S_0 je najprv naplnený pomocou aktuálneho stavu sveta a potom je mu vygenerovaných 1 - 30 nových uzemnených literálov podľa Γ z DB.Tieto nové literály umožňujú pripravenie herného sveta na úlohu doplnením dočasných predmetov, nepriateľov či postáv. Podobne je vygenerovaných 1 - 10 literálov, ktoré tvoria cieľ G. Pre počiatočnú populáciu je takto vytvorených 100 jednotlivcov.

Po vygenerovaní počiatočnej populácie sú jednotlivci ohodnotení funkciou fitness. Táto funkcia ohodnocuje úlohy podľa toho, ako veľmi sa ich dej podobá na požadovaný dejový obluk. Dejový oblúk je jednoduchý vzor, ktorý popisuje štruktúru literálneho diela. Najznámejší dejový oblúk je dejový oblúk s trojdejstvovou štruktúrou (Obr. 2.3), ktorý sa používa vo filmovom priemysle. Tento dejový oblúk rozdeľuje dej na prípravu (1/4 príbehu), konfrontáciu (2/4 príbehu) a rozuzlenie (1/4 alebo menej). Táto štruktúra má tri významné body: 1. dejový zvrat (spúšťajúca udalosť), 2. dejový zvrat (bod v deji kedy je neisté, či hrdinovia vyhrajú alebo budú porazení) a kríza (vyvrcholenie deja, po ktorom napätie v príbehu opadá).



Obr. 2.3: Dejový oblúk s trojdejstvovou štruktúrou
Zdroj: Preložené do slovenčiny podľa obr. 4 z [4].

Pri hodnotení úlohy postupujeme nasledovne. Najprv použijeme plánovač na získanie plánu (deja) úlohy. Musíme ošetriť prípad, keď plánovač nenájde riešenie. Vtedy má úloha prázdne riešenie (čo spôsobí nízke hodnotenie). Následne sa pozrieme do množiny E z DB, ktorá každej akcii priraďuje jej efekt na napätie deja. Ak má úloha plán p tak jej dejový oblúk môžeme symbolicky zapísať ako $p_{arc}^{sym} = (s_1, s_2, ..., s_n)$. Ako

príklad si vezmeme nasledovný plán:

`attack(zombie, anne, home), go(john,johnhome, village), go(john, village,store), get(john, antidote1, store), go(john, store, village), go(john, village, johnhome), cure(john, anne, antidote1, johnhome)`

Ak potom $E = \{$ `attack` : $+$, `get` : $+$, `go` : $=$, `cure` : -, ... $\}$, tak dejový oblúk pre tento plán je $p_{arc}^{sym} = (+, =, =, +, =, =, -)$. Dejový oblúk môžeme zapísať aj číselne ako $p_{arc}^{num} = (v_1, v_2, ..., v_n)$, kde $v_0 = 0$ a v_i získame pomocou funkcie:

$$v_i = \begin{cases} v_{i-1} + 1, & \text{ak} s_i = \texttt{+}, \\ v_{i-1} - 1, & \text{ak} s_i = \texttt{-}, \\ v_{i-1}, & \text{ak} s_i = \texttt{=}. \end{cases}$$

Dejový oblúk z príkladu v tomto zápise je $p_{arc}^{num} = (1, 1, 1, 2, 2, 2, 1)$. Takto získaný dejový oblúk potom funkcia fitness porovná s požadovaným dejovým oblúkom (zadáva sa ako parameter, v práci bol použitý dejový oblúk z obr. 2.3). Skóre je tým vyššie, čím je dejový oblúk úlohy podobnejší požadovanému dejovému oblúku. Autori práce veria, že týmto spôsobom je možné generovať úlohy, ktorých dej sa postupne stupňuje až do konca, kedy úloha dospeje k záveru.

Keď sú všetci jedinci ohodnotení, je potrebné vybrať tých, ktorý sa stanú rodičmi novej populácie. Autori sa rozhodli zvoliť prístup, kde na základe výsledkov funkcie fitness sa vyráta pravdepodobnosť, že sa jednotlivec stane rodičom. To znamená, že jednotlivci pre reprodukciu sú volený náhodne, ale tí s lepším hodnotením majú na zvolenie lepšiu šancu. Zároveň, aby sa zabránili zhoršovaniu kvality jedincov, niekoľko najlepšie ohodnotených úloh bude prekopírovaných do nasledujúcej generácie bez zmeny.

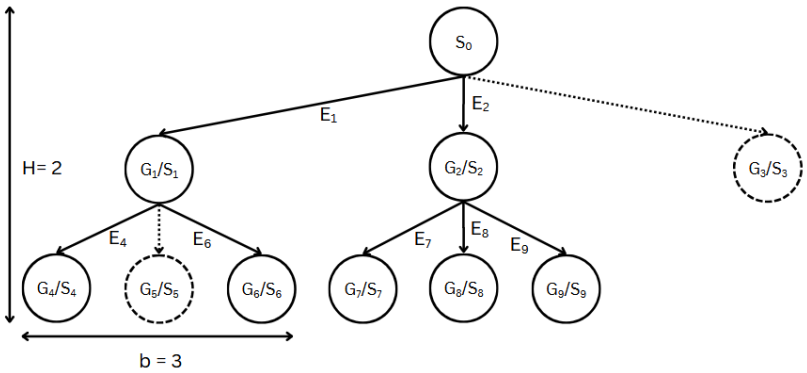
Keď sú zvolení dvaja jednotlivci na reprodukciu, prebehne ich kríženie. S_0 a G oboch týchto jedincov sa náhodne rozdelia na dve časti (nemusia obsahovať rovnaký počet literálov). Následne sa vytvoria dvaja noví jedinci pričom prvý z týchto jedincov má prvú polovicu svojho počiatočného stavu a cieľa od jedného rodiča a druhú od druhého rodiča. Druhý potomok dostane prvú polovicu od druhého rodiča a druhú od prvého. Počas vytvárania týchto potomkov, algoritmus automaticky odstráni duplicitné literály a zabezpečí, aby vzniknutí potomkovia dodržiavali obmedzenia z Γ z DB. Následne prebieha mutácia vzniknutého potomka, kedy sa náhodne pridá alebo odstráni (alebo oboje) jeden literál z S_0 alebo G (alebo z oboch). Pri pridávaní nového literálu sa nejaký vygeneruje podľa doménovej databázy. Proces reprodukcie skončí, keď nová generácia obsahuje 100 jednotlivcov. Vtedy sa z tejto generácie stáva aktuálna populácia, ktorá je znovu ohodnotená a celý proces sa opakuje dokým nie je vytvorených 100 generácií.

Metóda bola otestovaná pomocou jednoduchej 2D RPG hry, do ktorej bola začlenená navrhnutá architektúra tak, aby hra využívala úlohy vytvorené genetickým

algoritmom. Genetický algoritmus vytvoril pre hru tri úlohy a ďalšie tri úlohy v hre boli vytvorené ľudským tvorcom. Následne bol vykonaný test podobný Turingovmu testu, kde 34 respondentov hralo vzniknutú hru a po dokončení každej úlohy mali rozhodnúť, či táto úloha bola vytvorená algoritmom alebo človekom. Výsledky ukázali, že respondenti dokázali správne rozhodnúť zhruba v polovici prípadov, čo naznačuje, že vygenerované úlohy sú veľmi podobné úlohám od ľudských autorov. Ďalšie testovanie porovnávalo úlohy vytvorené genetickým algoritmom s náhodne generovanými úlohami pomocou funkcie fitness. Genetický algoritmus prekonal náhodne generované úlohy už po pár generáciách. Treba však dodať, že genetický algoritmus je výpočtovo náročný a preto sa celá dejová línia musí vygenerovať pred začatím hry. Hráč už iba prechádza cez pripravenú lineárnu postupnosť úloh, ktorú musí dodržať.

2.4 Generovanie úloh s viacerými vetvami riešení

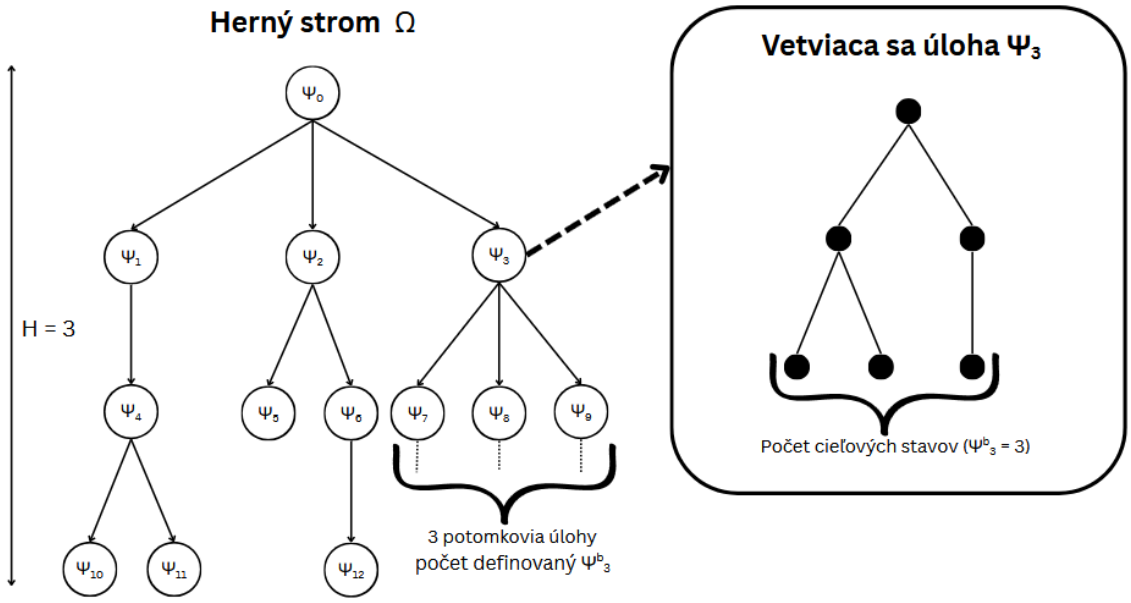
Na prácu popísanú v predchádzajúcej sekcii nadväzuje práca [2], ktorá predstavuje metódu, ktorá s použitím genetického algoritmu dokáže generovať nelineárne úlohy s vetviacim sa dejom. Takéto úlohy majú hráčovi umožniť riešiť ich viacerými spôsobmi a na základe zvoleného riešenia bude hráčovi prezentovaná nová úloha. Jedna možná vetva deju v nelineárnej úlohe je v práci označená ako dejová línia. Nelineárne úlohy sú reprezentované pomocou stromovej štruktúry, kde koreň predstavuje počiatočný stav herného sveta S_0 úlohy a listy stromu predstavujú možné cieľové stavy úlohy G_i. Vnútorné vrcholy definujú prechodné stavy. Každý takýto prechodný stav je súčasne cieľovým stavom (G_i) predchádzajúcej časti úlohy a počiatočným stavom (S_i) nasledujúcej časti úlohy. Jedna vetva stromu (E_i) pozostáva z dvoch stavov (S_j, G_i) a hrany ($S_j \rightarrow G_i$), ktorá zahŕňa postupnosť akcií potrebných na dosiahnutie prechodného alebo cieľového stavu G_i. Príklad štruktúry takejto úlohy je zobrazený na obrázku **??**.



Obr. 2.4: Príklad štruktúry úlohy s vetviacim sa dejom s výškou H = 2 a vetviacim faktorom b = 3. Čiarkované kruhy a šípky predstavujú neplatné vetvy, ktoré nemajú riešenie.

Zdroj: Vlastný obrázok podľa obr. 4 z [2].

Podobne ako v [4] aj tu autori chceli zachovať logickú následnosť úloh a zohľadňovať v nasledujúcich úlohách zmeny v stave sveta, ktoré boli vykonané v predchádzajúcich úlohách. Keďže však jedna úloha má viacero dejových línií a každá dejová línia môže inak ovplyvniť herný svet, je potrebné pre každý koniec úlohy vygenerovať novú nasledujúcu úlohu. Takýmto generovaním vznikne stromová štruktúra - herný strom Ω, ktorý reprezentuje dej celej hry (obr. 2.5). Každý jeho vrchol Ψ_i, obsahuje jednu úlohu, ktorá ako sme už povedali je tiež stromová štruktúra. Počet potomkov vrcholu Ψ_i herného stromu, ktorý nie je list závisí od počtu dejových línií úlohy, ktorú obsahuje.



Obr. 2.5: Štruktúra deju hry reprezentovaná pomocou herného stromu
Zdroj: Preložené do slovenčiny podľa obr. 1 z [2].

Architektúra systému je rovnaká ako v časti 2.3 (obr. 2.1) a teda systém pozostáva z OQG a GM. Rozdiel je v tom, že tento krát musí genetický algoritmus v OQG vytvárať herný strom a nie len lineárnu postupnosť úloh. Keďže aj úlohy sú rozvetvené, jeden plánovací problém nebude reprezentovať celú úlohu ale len jednu jej vetvu E_i. Teda:

$$E_i = (P, S_j, G_i, O), i \prec j$$

P je množina atómov. O je množina akcií, ktoré sa dajú v hre vykonať a S_j a G_i sú počiatočný a cieľový stav hrany E_i. Prvky P a O sú rovnaké pre všetky vetvy v každej úlohe. Prvky S_j a G_i sú rôzne pre v každej vetve. Vyriešením tohto plánovacieho problému získame lineárnu postupnosť akcií, ktorá popisuje riešenie tejto vetvy úlohy.

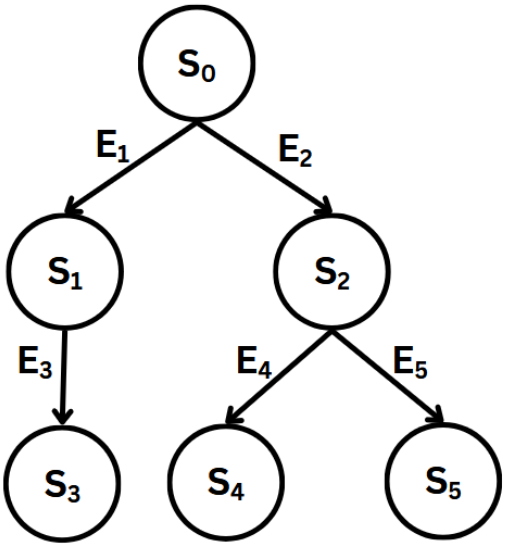
Vetvy pre počiatočnú populáciu genetického algoritmu sa generujú náhodne na základe informácií v doménovej databáze. Tá je podobná ako v časti 2.3. Líši sa len v množine obmedzení Γ, do ktorej pribudli tri nové typy obmedzení: dôležitosť opakovania, typy kvalifikácie a motívy cieľu. Pomocou dôležitosti opakovania vieme označiť,

ktorý parameter daného predikátu je dôležitý a ak sa tento predikát objaví v úlohe, opakované výskyty jeho dôležitého parametra v nasledujúcich častiach úlohy prispievajú k celkovej kontinuite úlohy. Typy kvalifikácie hovoria o tom, či daný predikát môže byť dynamicky pridaný do počiatočného alebo cieľového stavu úlohy bez toho, aby narušil logickú integritu a konzistenciu herného sveta. Predikáty označené kľúčovým slovom *initial* môžu byť pridané do počiatočného stavu úlohy (napr. dynamické vytvorenie dočasných predmetov či postáv pred začiatkom úlohy). Kľúčovým slovom *goal* sú označené tie predikáty, ktoré môžu byť použité na vytvorenie celového stavu úlohy. Motívy cieľu radia všetky predikáty do skupín podľa spoločného motívu. Napríklad predikáty *hasreceived* (postava dostala predmet) a $knowfaildtoget$ (postava vie, že iná postava nezískala predmet) majú oba motív *askitem* (postava žiadala o nejaký predmet). Predikáty s rovnakým motívom cieľu sú si tematicky podobné a preto sú dobrými kandidátmi na vytvorenie alternatívnych cieľov tej istej úlohy.

Proces vytvárania jednotlivca (jedna vetviaca sa úloha) pre počiatočnú populáciu genetického algoritmu je rozdelená do troch krokov. V prvom kroku sa náhodne zvolí zadávateľ úlohy spomedzi všetkých postáv, ktoré existujú v hernom svete. Zadávateľ je postava, ktorá hráčovi ponúkne úlohu, poskytne mu informácie potrebné na začatie úlohy a sľúbi odmenu v prípade, že hráč úlohu splní. Zadávateľ bude potom použitý pri generovaní jednotlivých vetiev ako hlavná postava v predikátoch tvoriacich prechodné a cieľové stavy. V druhom kroku je zvolený motív úlohy z pomedzi motívov v Γ. Používanie motívov zabezpečí, že vygenerovaná úloha bude pozostávať z podobných zadaní a teda sa nestane, že zadania jednotlivých vetiev spolu nebudú súvisieť. Posledným krokom je vygenerovanie samotného stromu reprezentujúceho úlohu. Generovanie začína od koreňa (počiatočný stav S_0). Pre každý vrchol v strome sa vygeneruje b nových vetiev E_i. b je parametrom algoritmu, ktorý definuje vetviaci faktor stromu. Plánovací problém $E_i = (P, S_j, G_i, O)$ každej vetvy je definovaný podľa doménovej databázy podobne ako v časti 2.3. Pri generovaní cieľového stavu G_i je využitý zadávateľ úlohy a zvolený motív na vytriedenie možných uzemnených literálov, ktoré môžu tvoriť cieľový stav. Do cieľového stavu sa môžu dostať len predikáty zo zvoleným motívom a hlavnou postavou v literáloch je vždy zadávateľ. Keď sú všetky vetvy aktuálneho vrcholu vyriešené, vyrieši sa plánovací problém pre každú jednu z nich. Ak sa pre niektorú vetvu nepodarí nájsť riešenie, vetva je odstránená z úlohy. Ak sa riešenie našlo, tak sa cieľový stav tejto vetvy môže stať prechodným stavom a môže byť použitý ako počiatočný stav pre ďalšie vetvy úlohy. Generovanie vetiev je rekurzívny proces a skončí vtedy, keď listy v strome úlohy dosiahnu definovanú výšku H.

Funkcia fitness rovnako ako v časti 2.3 hodnotí úlohy podľa toho, ako dobre zodpovedajú zvolenému dejovému oblúku. Zvolený dejový oblúk je na obrázku 2.3. Tento krát však nehodnotí jednoduchú postupnosť akcií, ale musí ohodnotiť celý strom, ktorý úlohu tvorí. Ohodnotenie takejto úlohy sa robí v štyroch krokoch. Najprv funkcia musí

prejsť celú stromovú štruktúru a získať všetky dejové línie, ktoré v danej úlohe môžu nastať. Na obrázku 2.6 je príklad úlohy p, ktorá má päť vetiev. V tejto úlohe existujú tri dejové línie $p_1 = E_1 + E_3$, $p_2 = E_2 + E_4$ a $p_3 = E_2 + E_5$. Po získaní týchto dejových línií funkcia fitness spojí postupnosti akcií na hranách jednotlivých vetiev tvoriacich jednu dejovú líniu do jednej sekvencie. Na základe týchto sekvencií a informácií v T z DB určí dejový oblúk pre každú dejovú líniu. Následne porovná každý dejový oblúk z požadovaným dejovým oblúkom a vyráta priemernú podobnosť medzi týmto dejovým oblúkom a úlohou. Na záver je vyrátaná zdatnosť jednotlivca na základe jeho priemernej podobnosti a ďalších faktorov hodnotenia, ktoré môžu byť použité na uprednostnenie niektorých úloh. Tieto doplňujúce faktory môžu byť priemerná dĺžka dejových línií (uprednostnenie dlhších/kratších úloh) alebo celkový počet vetvení. Podľa autorov je najvýznamnejší faktor kontinuity. Do kontinuity prispieva opakovanie parametrov, ktoré boli v Γ označené dôležitosťou opakovania. Kontinuita sa ráta tak, že prechádzame strom úlohy a vždy keď nájdeme nový dôležitý parameter, budeme počítať koľko krát sa vyskytol v nasledujúcich vetvách. Získané číslo potom dosadíme do výpočtu celkovej zdatnosti.



Obr. 2.6: Príklad štruktúry rozvetvenej úlohy p
Vlastný obrázok podľa obr. 7 z [2].

Keď máme úlohy ohodnotené, je čas vybrať rodičov ďalšej generácie a podrobiť ich kríženiu a mutácii. Ak máme vybraté dve úlohy P_1 a P_2 ako rodičov, najprv nájdeme v nich dvojice kompatibilných stavov. Označme P_kS_i stav S_i v úlohe P_k. Pár stavov z dvoch rodičov (P_1S_i a P_2S_j) je kompatibilný ak:

1. Oba stavy pozostávajú z rovnakých literálov ($P_1S_i = P_2S_j$). V takomto prípade všetky vetvy začínajúce v P_1S_i môžu začínať aj v P_2S_j a naopak bez narušenia logickej súdržnosti úlohy.

2. P_1S_i je podmnožinou P_2S_j. V tomto prípade vetvy z P_1S_i sa môžu objaviť v P_2S_j, ale vetvy z P_2S_j nemôžu byť prekopírované do P_1S_i.

3. P_1S_i je nadmnožinou P_2S_j. Táto možnosť je obdobná k predchádzajúcej. Vetvy z P_2S_j môžu byť prekopírované do P_1S_i, ale naopak to nie je možné.

Keď sú dvojice kompatibilných stavov identifikované, začne prechádzať cez stromy oboch rodičov P_1 a P_2, aby sme vytvorili dvoch nových potomkov C_1 a C_2. Stavy, ktoré nie sú kompatibilné zo žiadnym iným stavom sú prekopírované do potomka bez zmeny (nekompatibilné vrcholy z P_1 idú do C_1 a z P_2 idú do C_2). Kríženie sa deje na každej dvojici kompatibilných vrcholov. Ak máme kompatibilnú dvojicu 1. typu, tak každá vetva z P_1S_i a P_2S_j má rovnakú šancu, že sa skopíruje na príslušné miesto buď do C_1, alebo do C_2. V prípade 2. typu každá vetva z P_1S_i má 50% šancu že sa skopíruje do stavu P_2S_j v potomkovi C_2. Ak sa neskopíruje, tak na jej miesto príde pôvodná vetva z P_2S_j (podľa poradia, v ktorom sú vetvy usporiadané v P_1S_i a P_2S_j). Vetvy v potomkovi C_1 sú priamo prekopírované z P_1S_i. V 3. prípade môžu byť rovnakým spôsobom vetvy z P_2S_j prekopírované do C_1.

Mutácia vytvorených potomkov prebieha tak, že vezmeme jednu úlohu a v nej môžeme náhodne zvoliť vrchol. Tento vrchol je potom buď odstránený z úlohy, alebo sa pokúsime vygenerovať nové vetvy (podľa DB), ktoré budú začínať v tomto vrchole.

Výsledný systém bol otestovaný pomocou jednoduchej 2D RPG hry, ktorá implementovala niekoľko úloh vytvorených algoritmom a niekoľko vytvorených ľudských dizajnérom. Herný strom Ω bol navrhnutý tak, aby každá dejová línia hry obsahovala práve dve vygenerované úlohy a jednu vytvorenú človekom. Bol vykonaný test podobný Turingovmu testu, kde 38 hráčov hralo túto hru. Každý z nich prešiel práve jednou ľubovoľne zvolenou dejovou líniou hry a po dokončení každej úlohy rozhodol, či danú úlohu pokladal za generovanú alebo vytvorenú človekom. Hráči správne určili typ úlohy v zhruba v polovici prípadov z čoho sa dá usúdiť, že je ťažké rozlíšiť úlohy vytvorené algoritmom od tých navrhnutých dizajnérom. V druhom testovaní boli pomocou funkcie fitness porovnané úlohy vytvorené genetickým algoritmom s náhodne generovanými úlohami. Úlohy z genetického algoritmu prekonali náhodné úlohy už po pár generáciách a po 35 generáciách začal genetický algoritmus konvergovať k optimálnemu riešeniu problému.

Na záver by sme chceli dodať, že hoci tento systém umožňuje hráčovi riešiť zadané úlohy rôznymi spôsobmi a tým doviezť hru do rôznych koncov, nemusí mu nutne poskytnúť pocit, že jeho akcie majú zásadný vplyv na herný svet. Úlohy sú síce vytvárané tak, aby medzi nimi nevznikali logické nezrovnalosti zároveň ich však nič nenúti, aby spolu súviseli, alebo na seba tematicky nadväzovali. To môže viesť k tomu, že celkový príbeh hry bude málo pútavý a v hráčovi to môže vzbudiť pocit, že jeho akcie nemajú žiadne následky a dej hry sa nikam neposúva.

Kapitola 3

Ciele práce

Cieľom našej práce je ...

Kapitola 4

Metodika

Kapitola 5

Návrh a implementácia

Kapitola 6

Výskum

Kapitola 7

Výsledky a diskusia

Kapitola 8

Záver

Bibliografia

[1] Vincent Breault, Sébastien Ouellet a Jim Davies. „Let CONAN tell you a story: Procedural quest generation“. V: *Entertainment Computing* 38 (2021), strana 100422. ISSN: 1875-9521. DOI: https://doi.org/10.1016/j.entcom.2021.100422. URL: https://www.sciencedirect.com/science/article/pii/S1875952121000197.

[2] Edirlei Soares de Lima, Bruno Feijó a Antonio L. Furtado. „Procedural generation of branching quests for games“. V: *Entertainment Computing* 43 (2022), strana 100491. ISSN: 1875-9521. DOI: https://doi.org/10.1016/j.entcom.2022.100491. URL: https://www.sciencedirect.com/science/article/pii/S1875952122000155.

[3] Jonathon Doran a Ian Parberry. „A prototype quest generator based on a structural analysis of quests from four MMORPGs“. V: *Proceedings of the 2nd International Workshop on Procedural Content Generation in Games*. PCGames '11. Bordeaux, France: Association for Computing Machinery, 2011. ISBN: 9781450308724. DOI: 10.1145/2000919.2000920. URL: https://doi.org/10.1145/2000919.2000920.

[4] Edirlei Soares de Lima, Bruno Feijo a Antonio L. Furtado. „ Procedural Generation of Quests for Games Using Genetic Algorithms and Automated Planning “. V: *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*. Los Alamitos, CA, USA: IEEE Computer Society, okt. 2019, strany 144–153. DOI: 10.1109/SBGames.2019.00028. URL: https://doi.ieeecomputersociety.org/10.1109/SBGames.2019.00028.

[5] Alex Stocker a Chris Alvin. „Non-Linear Quest Generation.“ V: *FLAIRS*. 2018, strany 213–216.

[6] Susanna Värtinen, Perttu Hämäläinen a Christian Guckelsberger. „Generating Role-Playing Game Quests With GPT Language Models“. V: *IEEE Transactions on Games* 16.1 (2024), strany 127–139. DOI: 10.1109/TG.2022.3228480.

Dodatok A

Zdrojové kódy, dáta